vasic-digital tmux — Open Issues Tracker

> **Canonical source of truth for everything currently unfinished, partially validated, or at risk of violating the anti-bluff covenant (`Constitution.md` §1 + §11.4.1 through §11.4.6).**
>
> Every PASS in this codebase MUST carry positive evidence captured live that the feature works for the end user. Metadata-only PASS, configuration-only PASS, "absence-of-error" PASS, and grep-based PASS without runtime evidence are all critical defects regardless of how green the summary line looks. **Tests AND HelixQA Challenges are bound equally.**
>
> Forensic anchor — direct user mandate (verbatim, 2026-04-28 + 2026-05-07 + 2026-05-08, repeatedly reasserted from upstream `vasic-digital` projects):
>
> > "We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"
>
> §11.4.6 forensic anchor (verbatim, 2026-05-08):
>
> > "'LIKELY' is guessing, we MUST NOT have guessing, since it can be or may not be! No bluffing and uncertainity is allowed at any cost! We MUST always know exactly precisly what is happening exactly, in any context, under any conditions, everywhere!"

Compiled: 2026-05-08 (Phase B per-session-containerization cycle). **Author:** Engineering coordinator **Working pool source:** the items below feed the active task list directly. Each item carries a current state, the captured-evidence requirement, and a fix-direction proposal so future-self can resume cold.

> **Migration policy** (mirrors upstream `vasic-digital` projects): resolved items are moved to **`Fixed.md`** in the same commit that closes them. **`Issues.md` holds OPEN / PARTIAL / BLOCKED / RUNNING / INVESTIGATED only**; once an item is closed and verified end-to-end, it migrates to `Fixed.md` and disappears from here. Never delete items outright — history matters for cold-start handover.

Document conventions

Code	Meaning
`OPEN`	Unfinished work; needs implementation + anti-bluff coverage
`PARTIAL`	Implementation exists but coverage gaps remain (positive evidence missing or environmental SKIP unresolved)
`BLOCKED`	Cannot progress without external dependency (host capability, third-party tool, distro support, etc.)
`RUNNING`	In-flight in this session (background process, ongoing test cycle)
`INVESTIGATED`	Forensic investigation produced findings; closure pending decision on fix scope

Status reclassification rules (§11.4.6 enforced):

- A `PARTIAL` may not move to closed without runtime evidence (`/sys/fs/cgroup/.../memory.max` readback, `systemctl status` showing scope active, `kill -9` survivor proof — never just script exit code).
- A `BLOCKED` reclassifies to `OPEN` when the blocking dependency resolves; it must NOT skip directly to closed.
- An `INVESTIGATED` item must record the captured forensic trace (file path / command output / log timestamp) — never speculation ("likely" / "probably" / "appears to" — see Constitution §11.4.6).

Categories:

- **A** — Tooling / harness gaps
- **B** — Anti-bluff completeness across the existing test surface
- **C** — Per-session containerization features pending evidence
- **D** — Host-capability + topology dispatch gaps
- **E** — Documentation / Continuation drift

A. Tooling / harness gaps

A2 RUNALL-NATIVE-RESOLVE-001 — standalone run_all.sh mis-resolves the binary on native macOS

Status: Fixed (→ Fixed.md) **Type:** Task

CLOSED v1.0.27. `scripts/tests/run_all.sh` hardcodes `TMUX_BIN=tmux/build/bin/tmux` (the `build_containerized.sh` output path) and is the containerized-build validator. On native macOS the authoritative validator is `setup.sh` (builds + verifies against `tmux/build-darwin/`). Invoked standalone on macOS with a stale `tmux/build/` present (a prior Linux containerized build), run_all.sh resolved the wrong-arch binary → `Exec format error` mass-FAIL (observed 2026-06-16; NOT a product defect — `setup.sh` run_all `55/0/6` + installed-binary smoke GREEN the same session; removing the stale `tmux/build/` then gave `not executable` because run_all expects that path). **Fix direction:** make run_all.sh OS-aware (prefer `tmux/build-darwin/` on Darwin) OR document that native-macOS validation is `setup.sh`-only and run_all.sh is the containerized path. Captured-evidence requirement: a clean native-macOS run_all GREEN after the fix.

A3. META-TEST-72-73-COVERAGE-001 — tests 72/73 need persistent meta-test mutations

TMX-ID: TMX-076 **Type:** Task **Status:** Ready for testing

Tests 72 (libevent/ncurses local-dependency obtain) and 73 (build_native.sh local-dependency wiring) currently have no paired §1.1 mutation registered in scripts/tests/meta_test_false_positive_proof.sh, so a regression in either mechanism would not be mechanically caught by the layer-4 anti-bluff sweep. Re-filed follow-up from TMX-071 (originally tracked 2026-06-30 in commit `8232b15`, reverted the same day in commit `9b719a6` purely because of an unrelated add→db-to-md rendering tooling defect, not because the underlying work was invalid or completed — confirmed via git history per §11.4.124; the mutation-writing itself was never done). Acceptance: two new persistent mutations (mirroring the existing M-test67/CM-LOCAL-DEPS-MECHANISM pattern) that mutate a real invariant each test enforces and assert the harness FAILs, then restore cleanly; both wired into the standing sweep alongside the existing 60+ mutations.

A4. NEW-COLLISION-GUARD-SCOPE-GATED-001 — the `new` verb's Linux collision guard is skipped entirely when systemd user-scopes are unavailable

TMX-ID: TMX-077 **Type:** Bug **Status:** Ready for testing

Found by the final whole-branch review of the wizard/password redesign (2026-07-05), confirmed pre-existing (not introduced by that plan) via direct inspection of scripts/tmx.template around the `new` verb's collision check. On Linux, `tmx new -s NAME` refuses to proceed when a session already occupies NAME's systemd scope — but that refusal is gated by `[ "$_scope_ok" -eq 1 ]`, and `_scope_ok` is set to 0 whenever `systemctl --version` reports below 230, `systemctl` is entirely absent, or a real `systemd-run --user --scope` probe fails (see the topology probe a few lines above the guard). On any such host, the guard is skipped outright rather than falling back to an alternate collision check (the way Darwin uses raw socket presence instead), so `tmx new -s NAME` against an already-live session of the same name could proceed further than intended before tmux's own internal duplicate-session refusal (if any) is reached — including possibly reaching the password-collection prompt for what the operator believes is a brand-new session. Not yet reproduced end-to-end on a real scope-unavailable host (this project's dev/CI hosts all have a working systemd user session), so the exact downstream behavior (does tmux's own duplicate-session check catch it first, and if not, could a live session's persisted password state be disturbed) is UNCONFIRMED pending that reproduction. **Fix direction:** give Linux a scope-independent fallback collision check mirroring Darwin's socket-presence check (e.g. `tmux -L SOCK_LABEL has-session -t NAME` before proceeding) so the refusal does not depend on `_scope_ok` at all. **Acceptance:** a test that forces `_scope_ok=0` (or runs on a genuinely scope-less host) and asserts `tmx new -s NAME` against an already-live NAME still refuses cleanly, with no password prompt reached.

A5. ADD-HASH-CONVENTION-SPLIT-001 — `add.go` and `parser.go` compute heading_hash from two different conventions

TMX-ID: TMX-093 **Type:** Task **Status:** Queued

What. `computeHeadingHash(category, code, title)` is called with two different second arguments: `add.go:52` passes the ATM id (`TMX-093`) where `parser.go:201` passes the block code (`B1`). The same logical item

therefore hashes differently depending on which path created its row, so an `add`-created row never binds to the parser's hash for its own block.

How it manifests. A row created by `add` is reached by its declared `**TMX-ID:**` rather than by heading identity, so the first sync after an operator hand-writes that item's block performs an identity rebind: the row is updated, `identity rebinds: 1` is printed, and an `item_history` row is recorded. Surfaced by independent review 2026-09-01, which walked the full lifecycle and confirmed the behaviour is bounded — exactly ONE honest rebind, after which the hash lookup hits and the sync converges to `unchanged`. It is a standing surprise, not a data-loss path.

Reproduction. `workable-items add -title X -category B ...`, then hand-write the matching `### B<N> X` block declaring that id into Issues.md, then `sync md-to-db`: observe the `identity rebinds: 1` line. A second sync reports `unchanged`.

Acceptance. Either normalise the two call sites onto one convention (and migrate any existing add-created rows), or document the split at `computeHeadingHash` so the rebind is expected rather than surprising — whichever the operator prefers. Live corpus is unaffected today: sync reports `unchanged=90` with zero rebinds.

A6. ADD-ROWS-NOT-RENDERED-001 — `db-to-md` does not render rows created by the `add` verb

TMX-ID: TMX-094 **Type:** Bug **Status:** Queued

What. `sync db-to-md` replays `document_sources` reconciled against the structured rows, so a row that has no block in the source markdown is not rendered at all. A row created by `workable-items add` is exactly that: it exists in the SSoT and is invisible in the tracker documents.

How it manifests. The item is silently absent from `Issues.md` / `Fixed.md` and from their exports, so it does not reach an operator reading the trackers even though `validate` and the DB both hold it — the §11.4.148 integrity contract (an item must carry status, type and id on ALL surfaces) is not met for these rows.

Why this is filed now. The defect is PRE-EXISTING and is referenced in TMX-076 only as historical context for why TMX-071's work was reverted; no item tracks the rendering defect itself. Confirmed

untracked by grep over both trackers 2026-09-01 (with a control needle proving the search was not blind), and re-demonstrated by independent review the same day: an `add`-created row produced an empty rendered `Issues.md`. Filed so it stops being carried only inside another item's narrative (§11.4.197 — a known defect with no tracked item evaporates).

Reproduction. `workable-items add -title X ...` into a scratch DB, then `sync db-to-md --out-dir <tmp>`: the rendered `Issues.md` contains no block for X.

Acceptance. `db-to-md` emits a block for every structured row that has no corresponding source block (appended to its category section), so a round-trip after `add` is lossless; covered by a test that adds a row, renders, and asserts the block is present, with its §1.1 mutation removing the emit.

B. Anti-bluff completeness across the existing test surface

(Prior B-items closed: B3 P5-M20/P5-M21 escapes CLOSED in v1.0.16 [tests 49/50 + meta-test retarget], state-verified 2026-05-29 with `MUTATIONS CAUGHT 45 / ESCAPED 0`, migrated to `Fixed.md` §B3 as TMX-054; B1 CHAL-COVER-001, B2 TEST-AUDIT-001 also in `Fixed.md`. New open work below.)

C. Per-session containerization features pending evidence

(none open at this time — C1 TMX-T5, C2 TMX-T7, C3 TMX-T8 landed in `Fixed.md`.)

D. Host-capability + topology dispatch gaps

D2 TMPDIR-HARDCODE-001 — tests hardcoding /tmp false-FAIL under host disk-pressure

Status: Fixed (→ Fixed.md) **Type:** Task

CLOSED v1.0.27. Several tests create scratch under a hardcoded `/tmp` (e.g. `27_state_persistence.sh` target `tmx-test-18-target-*`). When the host root volume is full (observed 2026-06-16 on macOS, `/` at <200 MiB during the operator's away-window), the `cd`/mkdir into `/tmp` fails → `pane_current_path=''` false-FAIL instead of an honest §11.4.3 SKIP-with-reason. The tests pass normally + standalone (27 `3/3`, 38 `3/3`, 43 `15/0` re-run the same session) — the failure is purely the abnormal host-disk condition (a §11.4.1 FAIL-bluff class: environment, not product). **Fix direction:** route test scratch through `$TMPDIR` (operator now sets `/Volumes/T7/tmp` via ~/.zshrc + ~/.bashrc) OR guard each test on `/tmp` writability and SKIP-with-reason (§11.4.3/§11.4.50). Captured-evidence requirement: an induced-disk-full run shows SKIP-with-reason, not FAIL.

E. Documentation / Continuation drift

(CONTINUATION.md §3 entries that resolve land in `Fixed.md` per Constitution §5 / §12.10. New open work below.)

F. Runtime crash — operator-gated reproduction

(none open — F1 `tmx` session named "HelixCode" crashes the whole terminal RESOLVED 2026-06-13, operator-confirmed, migrated to `Fixed.md` A45 with reproduced root cause [stale wrapper `TMUX_BIN` pointing at a non-existent prior-checkout path → `exec` of missing binary → login shell dies → terminal closes] + 4-layer regression guard [test 60 + verify gate `CM-TMX-WRAPPER-TMUXBIN-VALID` + meta `M-WRAPPER-TMUXBIN`]).

Last reviewed: 2026-06-19 (v1.0.27 WIP burn-down. §11.4.138 operator-escape A51 [name:color prompt rejection] FIXED — shell-init + SSH-dispatcher char-set now allows `:` and `#`, max-length 64→80; test 65 operator-escape guard PASS=6/6. M24/A2/D2 in parallel streams per §11.4.103.)

M24-ESCAPE-001 — meta-test M24 (hostname 4-surface color) escapes: test 26 misses a 3-set-line removal

Status: Fixed (→ Fixed.md) **Type:** Bug **Severity:** Minor (test-coverage gap, no user-facing break — closed v1.0.27: `count=1` removed from M24 regex → strips from BOTH `_apply_color` and `_apply_host_color`, test 26 now FAILs on the mutation → CAUGHT) **Closure:** meta-test 37 CAUGHT / 0 ESCAPED (was 34 CAUGHT / 3 ESCAPED pre-fix)

What: The §1.1 paired-mutation `M24` in `scripts/tests/meta_test_false_positive_proof.sh` removes three of the four `tmux set -g …` lines in `_apply_host_color()` and expects test 26 to FAIL. The harness reports `MUTATION ESCAPED` — test 26 does not FAIL, because it asserts only a subset of the four surfaces, so removing the un-asserted set-lines leaves the test green.

Evidence: `bash scripts/tests/meta_test_false_positive_proof.sh 2>&1 | grep M24` → `FAIL: M24: MUTATION ESCAPED — test 26 did not FAIL with the three set-lines removed`. Confirmed pre-existing (M24 added in commit `f151d13`, v1.0.9 — before the per-session-color feature). Surfaced 2026-06-19 during the per-session-color M25/M26 verification run.

Fix direction: strengthen test 26 to assert ALL FOUR surfaces (`status-style`, `pane-active-border-style`, `clock-mode-colour`, `window-status-current-style`) so the M24 mutation (removing any 3) reliably FAILs it — mirroring the all-4-surfaces assertion already proven in test 63 T3 for the per-session path. (This also closes the symmetry gap: the per-session path has a 4-surface guard; the hostname path should too.)

Out of scope: the per-session-color feature (TMX-051). Tracked separately so the color release is not blocked by an unrelated pre-existing test-gap.

G. Interactive wizard + session-password redesign (2026-07-05)

New OPEN work from the 14-task wizard + session-password redesign plan (`docs/superpowers/plans/2026-07-05-tmx-wizard-password-redesign.md`; spec `docs/superpowers/specs/2026-07-05-tmx-wizard-password-redesign-design.md`). Code lands across sibling tasks of that plan; these four entries track the four user-visible requirements from the operator mandate (random-suffix create, masked password input, single-prompt reopen, existing-session picker).

Section closed 2026-09-01. All five entries have landed and now live in `Fixed.md`: G1/TMX-072, G2/TMX-073, G3/TMX-074, G4/TMX-075 (the four listed above) and the session-name sanitization work as §A55/TMX-078. Their blocks were duplicated here while they were also in `Fixed.md`; the duplicates were retired on operator decision so each item has exactly one block. The heading is kept for the historical context above — it is deliberately empty of items, not missing them.

H. Pre-existing timing issues surfaced during the 2026-08-10 no-limits-by-default cycle

Discovered while validating TMX-079 (see `Fixed.md`). Confirmed via §11.4.114 A/B isolation against the v1.0.38 baseline (identical failures reproduced BEFORE any of TMX-079's changes were applied) — NOT caused by TMX-079, and TMX-079's own fix + tests are unaffected. Tracked here so they are not lost.

H1 STATE-HOOK-RACE-001 — test 27 sub-check "18" (run-shell cwd-record hook) intermittently fails to observe the hook's write

TMX-ID: TMX-080 **Type:** Bug **Status:** Reopened

Reopened-Details: By: AI. On: 2026-08-13. Reason: cycle-re-discovered. Evidence: dedicated `/superpowers:systematic-debugging` pass (this cycle) — see below. This is a correction of the ORIGINAL diagnosis, not a new defect: the underlying failure is the same one first observed 2026-08-10, but this investigation disproves its central claim ("iteration

1 specifically, deterministically") and supersedes it with corrected, more complete evidence. Reopened per §11.4.34 rather than left as `Queued` because the entry's OWN content materially changed.

What (CORRECTED — the "iteration 1 specifically, deterministically" framing in the original 2026-08-10 report was WRONG): `scripts/tests/27_state_persistence.sh`'s sub-check labelled "18" drives the wrapper's `run-shell "$STATE_BIN record $SESS #{pane_current_path}"` hook directly (the same command the wrapper installs on `client-detached`/`session-closed`) and polls `tmx-state-bin recall` for up to 5 s waiting for the write to land. A dedicated instrumented investigation this cycle (adding timestamped diagnostic prints around the fire/kill/poll sequence, run 8× standalone across two separate probe sessions) shows the failure is **genuinely intermittent and NOT tied to any specific iteration number** — one run failed on iteration 2, a separate run failed on iteration 3, and several runs passed cleanly 3/3. The original report's "iteration 1, deterministically, 3/3 isolation runs" was based on an insufficient sample (apparently 1-3 trials that happened to land on iteration 1 by chance) and is corrected here.

Evidence (this cycle, 2026-08-13):

- 8 fresh, timestamped standalone runs: 2 showed a genuine FAIL, landing on iteration 2 and iteration 3 respectively (never iteration 1) — `FAIL 18 iter=2: ...` and `FAIL 18 iter=3: ...`, both with `poll_iterations_used=25` (the FULL 5 s poll elapsed and the state file NEVER updated to the hook's value — not merely late, genuinely never-written within the bound).
- On a failing iteration, `recall` immediately after `run-shell` fires AND immediately after `kill-session` returns BOTH already show the stale (pre-hook) value — the loss is not something the 5 s poll would have caught even with a longer bound.
- Root-cause CANDIDATE, partially confirmed: the wrapper's `kill-session` verb calls `systemctl --user stop "$SCOPE_UNIT"` immediately after `tmux kill-session`; the scope's `KillMode=control-group` (confirmed via `systemctl --user show`) SIGTERMs EVERY process in the cgroup, including an in-flight `run-shell`-spawned async child (confirmed via direct process-tree observation: the child DOES land in the SERVER's own scope, in both shared AND split topology, never the workload slice). A direct reproduction with an artificially-slowed hook command (`sleep 0.05; tmx-state-bin record ...`) reliably loses the write when immediately followed by `systemctl stop`, and reliably SUCCEEDS when `systemctl stop` is skipped entirely — confirming this mechanism is real and NOT a

test artifact (a real operator running `tmx kill-session -t NAME` is exposed to the identical race).

- **However, a first fix attempt (delaying `systemctl stop` until the scope's cgroup is observed empty, via a new `_wait_scope_quiescent` helper in `scripts/tmx.template` + an analogous change in `scripts/tmx-recycler.sh`) did NOT resolve the test's actual failure — it made it WORSE (5/5 reproducible failures, always landing on iteration 1, up from the original ~20-25% intermittent rate).** A corrected version of the same fix (resolving the cgroup path ONCE rather than re-querying `systemctl show` on every poll iteration, which itself has a race — `ControlGroup` reports empty almost immediately once the tracked main PID exits, before a still-running child has necessarily finished) ALSO failed to resolve the real test, despite BOTH fix attempts working correctly in isolated, simplified manual reproductions. Per the Iron Law (3+ fix attempts / worsening symptom → question the diagnosis, don't keep patching), **both fix attempts were reverted; `scripts/tmx.template` and `scripts/tmx-recycler.sh` are unchanged from before this cycle.**
- **A second, deeper anomaly surfaced and is NOT yet explained:** in the same instrumented investigation, `tmux display-message -t "$SESS" -p '#{pane_current_path}'` (what the test's own CD-landing poll checks, lines ~162-169) was observed to show STALE data (the Phase-1 target directory, never the freshly-`cd`'d hook target) in EVERY iteration of EVERY run tested — meaning that poll's OWN 5 s wait never actually succeeds and is effectively a no-op today — while a SEPARATE, immediately-following bare (`-t`-less) `run-shell "echo #{pane_current_path} > file"` diagnostic showed the CORRECT, freshly-`cd`'d value in some iterations and the SAME stale value in others, with no pattern yet identified tying this to the kill-timing race above. This suggests `#{pane_current_path}` may resolve differently (possibly via different tmux-internal code paths, or genuinely different cached/live state) depending on how it is queried, which the kill-session-timing hypothesis does NOT explain and needs its own investigation — most likely requiring direct study of tmux's own `#{pane_current_path}` resolution logic (not yet done; this cycle's investigation treated tmux as a black box via empirical probing only).

Fix direction (still not fully root-caused; genuinely more complex than originally scoped): at minimum TWO distinct phenomena are in play — (1) the `KillMode=control-group` race against an in-flight `run-shell` child (mechanism confirmed, but the attempted fix did not resolve the real test, meaning either the fix itself has a remaining bug, or this is not actually the dominant cause of the REAL test's failures

despite being real and reproducible in isolation), and (2) an unexplained discrepancy between `display-message`'s and a bare `run-shell`'s view of `#{pane_current_path}` for the same pane at nearly the same moment — **source-confirmed mechanism found for (2) this cycle: `tmux/osdep-linux.c`'s `osdep_get_cwd`/`osdep_get_name` resolve via `tcgetpgrp(fd)` (the PTY's CURRENT foreground process group at query time), not the shell's own tracked PID; if the pane's shell (oh-my-bash, per this operator's `.bashrc`) spawns prompt-render subprocesses that transiently become the foreground process group, different queries issued moments apart can read different processes' cwd.** Needs a fresh, dedicated systematic-debugging pass to CONFIRM this mechanism against a live failure (not yet done — see below) with either (a) a subagent given a clean context and unlimited focus on this single question, or (b) an experiment disabling oh-my-bash for test-created sessions to see if the divergence disappears. Out of scope to force a fix this cycle given the Iron Law guidance against continuing to patch a worsening symptom.

Cross-reference: TMX-081 (`Issues.md` §H2) is hypothesised to share this SAME root-cause mechanism (oh-my-bash prompt-render subprocess activity racing tmux's `tcgetpgrp`-based pane-state resolution), manifesting there as a CPU-throttle settle-window timeout rather than a stale-path read. See §H2 for the shared-cause hypothesis and the recommended confirming experiment.

H2 SPLIT-QUIET-SETTLE-001 — test 87 G4 ("quiet-phase control") occasionally fails to observe a zero-throttle settle window

TMX-ID: TMX-081 **Type:** Bug **Status:** Queued

What: `scripts/tests/87_server_scope_split.sh` G4 waits for a 1 s idle window with `nr_throttled` delta 0 on BOTH the server scope and the workload slice (settling past shell-startup noise) before G5 measures throttling under deliberate load. G4 fails ("quiet-phase never settled") with a small non-zero delta on the workload slice.

Evidence: reproduced identically on both the TMX-079-fixed tree and the v1.0.38 baseline (`git stash` A/B, 3× each, 2026-08-10) — `FAIL: G4: quiet-phase never settled (never-settled: .../tmxw-t87_<pid>.slice:delta=10)`. The isolation invariant G4 exists to set up for (G5: the server scope is NEVER co-throttled while the slice is)

PASSES consistently regardless, so the feature under test is proven sound — this is a control-window settle heuristic being slightly too strict/short, not a defect in the isolation mechanism itself.

Discovery note (this is the FIRST time it is tracked as its own item, hence `Queued` not `Reopened` per §11.4.34 — the latter presupposes a prior closure, and this defect was never previously closed): discovered by AI, 2026-08-10, during TMX-079 validation.

Fix direction (not yet investigated to root cause): needs either a longer settle window or a higher delta tolerance for the workload slice specifically (shell/pane-shim startup on a freshly-joined slice may legitimately cost a few throttled periods even at rest) — which of the two (or another cause entirely) is UNCONFIRMED without the actual `cpu.stat` samples across the settle window; needs a dedicated systematic-debugging pass (§11.4.102) before deciding — out of scope for TMX-079.

2026-08-13 investigation update (dedicated `/superpowers:systematic-debugging` pass — non-reproduction + a new, evidence-backed root-cause CANDIDATE, not a fix):

- **Could NOT reproduce today: 24/24 standalone runs of test 87 (4 individual + a 20-run background batch, with per-try `nr_throttled` deltas instrumented) all show G4 settling cleanly, typically on the FIRST 1 s window.** Per §11.4.7 (demotion requires evidence under the SAME conditions that originally exposed the defect), this does NOT close or downgrade TMX-081 — today's host conditions (this session has spun up and torn down a very large number of tmux sessions already, plausibly warming filesystem/package caches oh-my-bash's own startup checks read from) evidently differ from 2026-08-10's, and the defect is expected to still be real and reproducible under the original conditions (or under genuine host load / a cold shell-framework cache).
- **New root-cause CANDIDATE, source-confirmed (shared with TMX-080 — see H1's 2026-08-13 update): tmux's `#{pane_current_path}`/`#{pane_current_command}` (`osdep_get_cwd`/`osdep_get_name` in `tmux/osdep-linux.c`) resolve via `tcgetpgrp(fd)` — the PTY's CURRENT FOREGROUND PROCESS GROUP at query time — not the shell's own tracked PID.** If the pane's shell (this project's own `.bashrc`, confirmed to source oh-my-bash, is the operator's `$SHELL` used for `USER_SHELL` in `scripts/tmx.template`) spawns subprocesses as part of rendering its prompt (version-control status checks, `check_for_upgrade`, etc. — already independently documented as slow/complex by this project's own v1.0.41 test-58 fix), those

subprocesses transiently BECOME the foreground process group and consume REAL CPU time while doing so. Under G4's tight, quota-constrained cgroup (server+slice split to 50%/50% of an already-small `TMX_CPU` test budget), that subprocess activity can itself be CFS-throttled, extending well past the test's 12 × 1 s = 12 s settle bound if it runs long enough or is delayed enough by the quota.

- **This is a genuine CANDIDATE, not yet directly witnessed causing a G4 failure** — today's non-reproduction means the mechanism could not be confirmed live against an actual `nr_throttled`-non-zero window this cycle. It is offered as the most evidence-backed lead for whoever continues this investigation, not as a closure.

Cross-reference: TMX-080 (`Issues.md` §H1) is hypothesised to share the SAME underlying mechanism (oh-my-bash prompt-render subprocess activity racing tmux's `tcgetpgrp`-based pane-state resolution) manifesting as a DIFFERENT symptom — a stale/wrong `#{pane_current_path}` read there, vs. a settle-window timeout here. Confirming or refuting this shared-cause hypothesis (e.g., by testing whether disabling oh-my-bash for test-created sessions makes BOTH TMX-080 and TMX-081 stop reproducing) is the recommended next step for either investigation.

Fix direction (still not attempted — no live reproduction to validate a fix against this cycle): if the shared-cause hypothesis holds, candidate fixes include (a) a settle heuristic that specifically recognises oh-my-bash startup activity and excludes it from the "must be zero" requirement (e.g., waiting for `#{pane_current_command}` to stably show the shell itself, not a transient subprocess, before starting the settle-count), or (b) scoping test-created sessions to a minimal, non-framework shell so tests are not coupled to the operator's own interactive shell configuration. Needs a fresh reproduction (possibly requiring genuine host load, a cold oh-my-bash cache, or an artificially-injected slow prompt hook) before any fix can be validated per the project's own TDD-RED-first discipline (§11.4.43/§11.4.115) — forcing a fix without a live failing case to prove it against would itself be a bluff.

I. Live copy-mode wheel binding (2026-09-01)

Surfaced during the v1.0.44 verification work. Tracked here because it is a real, currently-failing check that was not previously recorded anywhere — leaving it only in a test log would be a §11.4.238 coverage escape at the tracking layer.

I1 WHEEL-COPY-MODE-OVERRIDE-001 — test 17 sub-check T3: the LIVE `WheelUpPane` binding is not the copy-mode override

TMX-ID: TMX-090 **Type:** Bug **Status:** In progress

What: `scripts/tests/17_scrollback_copy_mode.sh` sub-check **T3** reads the binding actually installed on the LIVE tmux server —

```
tmux -L "$S_SOCK" list-keys -T root WheelUpPane
```

— and requires the returned binding text to mention BOTH `copy-mode` and `scroll-up`. The project's `tmux.conf.template` deliberately OVERRIDES tmux's default `WheelUpPane` binding: the tmux default consults `#{mouse_any_flag}` and FORWARDS the wheel event to the running application, whereas the override enters copy-mode unconditionally so scrollback works even under mouse-tracking. T3 exists to prove the override is live on the server, not merely present in the config file.

Observed (current):

```
FAIL: T3: live WheelUpPane binding is not the copy-mode override
```

Status of the investigation: an investigation is IN FLIGHT. **The cause is NOT established.** No hypothesis is recorded here, because none has been confirmed against captured evidence — per §11.4.6 a cause may be stated only as a proven FACT or explicitly marked `UNCONFIRMED:` / `PENDING_FORENSICS:`, and a guess dressed as a lead is exactly what that clause forbids. What is known is only what is written above: the check reads the live server's binding, and on the current tree it does not match.

PENDING_FORENSICS: the `observed:` line the test prints immediately after the FAIL (`echo "  observed: $WHEEL_BIND"`) carries the ACTUAL binding text the live server returned. That output has not been captured into this entry. Capturing it is the first concrete step — it distinguishes at least three materially different situations that the FAIL

alone cannot: the override never reached the server (config not loaded / loaded from a different path), the override reached the server but in a form whose text does not contain both required tokens (a matcher-side problem in the test, i.e. potentially a §11.4.1 FAIL-bluff rather than a product defect), or the binding was subsequently replaced. Which of these holds is UNKNOWN until that output is read.

Relationship to other items: DISTINCT from TMX-080 / TMX-081 (§H1 / §H2) — those are timing/settle races around `#{pane_current_path}` and cgroup throttle windows. This one is a key-binding-presence check. No shared-cause hypothesis is asserted; whether one exists is UNCONFIRMED and would itself require evidence (§11.4.214 — this is a NEW id rather than a reopen precisely because no existing item describes this defect).

Note on a sibling check (not asserted as the same defect): `scripts/tests/47_alt_screen_scroll.sh` **T6** asserts a related live-`WheelUpPane`-override property. Whether T6 currently passes or fails on this tree has NOT been measured this cycle and is NOT claimed either way.

Fix direction: none proposed. Per §11.4.102 the systematic-debugging arc (reproduce → characterise → falsifiable hypothesis → fix against the PROVEN cause) must complete before any fix is written; proposing a direction now would be the guess-and-retry pattern that clause exists to prevent.